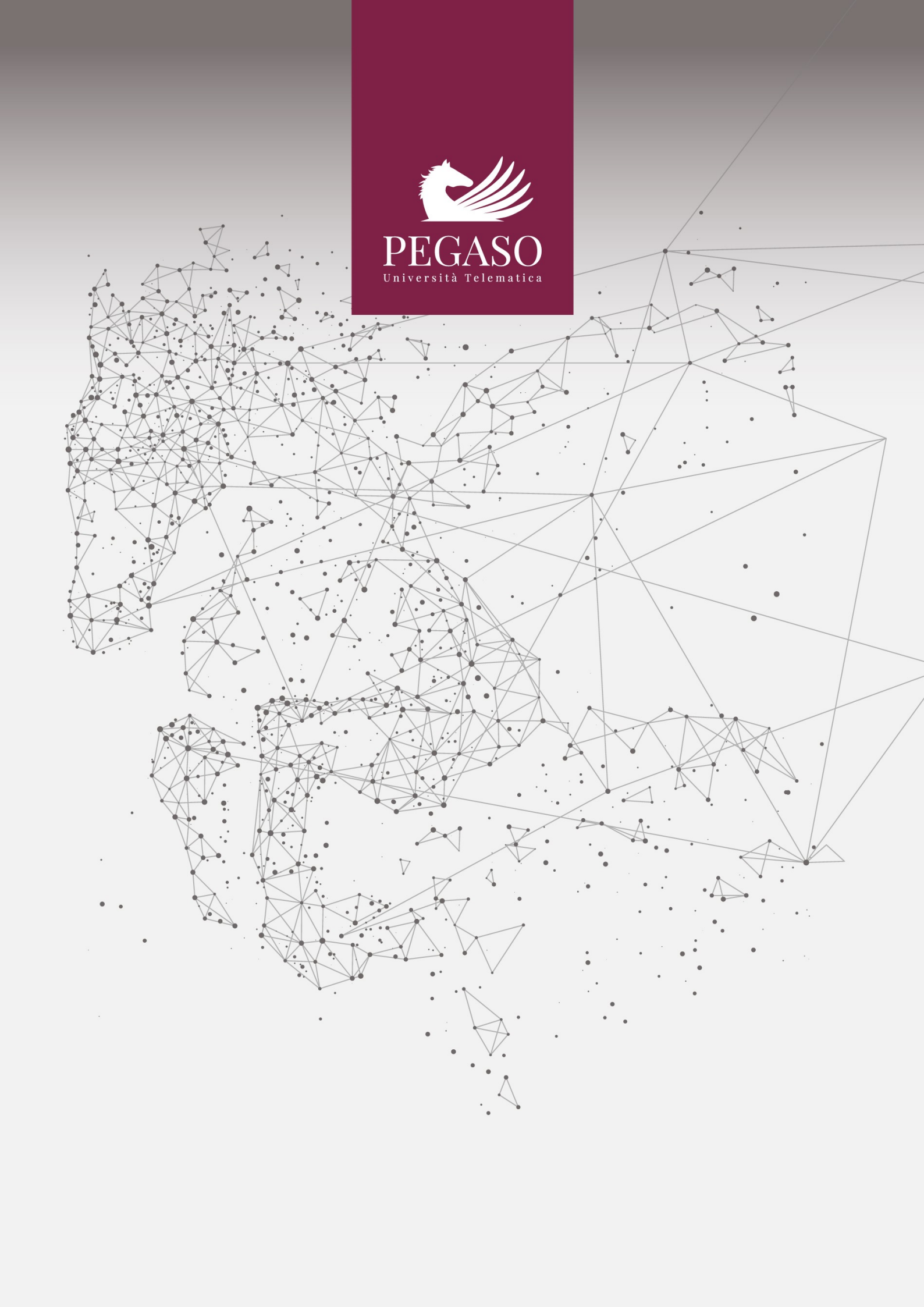

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. TIPI DI FILE 4

2.1 OPERAZIONI SUI FILE 4

3. ACCESSO AI FILE 6

4. STRUTTURE ITERATIVE PER L'ELABORAZIONE DEI FILE 12

BIBLIOGRAFIA 14

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano le operazioni di input e output da file. Si introduce una classificazione dei file e si inquadrano le operazioni possibili su di essi. Si illustra anche la differenza tra nome fisico e nome logico del file. Vengono forniti, inoltre, diversi esempi al fine di una maggiore comprensione di ciascuna delle operazioni per l'elaborazione dei file. Si forniscono, infine, le indicazioni per eseguire le operazioni di lettura da file attraverso le strutture iterative offerte da Python.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Tipi di file

I programmi realizzati finora richiedono che l'utente inserisca i dati ogni volta che il programma viene eseguito. Le variabili, infatti, vengono memorizzate in **memoria centrale** e quando il programma termina, lo spazio in memoria relativo alle variabili viene liberato.

C'è però la possibilità di salvare i dati in un file che viene registrato in memoria di massa.

Così facendo, diventa possibile recuperare tali dati anche in seguito alla terminazione di un programma.

In generale, esistono **due** tipi di file. I **file di testo** contengono i dati codificati come testo (solitamente si utilizza la codifica ASCII). Nella codifica ASCII ad ogni carattere è associato un numero compreso tra 0 e 255 e i numeri memorizzati nei file di testo corrispondono alle cifre simboliche e non ad un vero e proprio numero.

I **file binari**, invece, contengono dati che non sono stati convertiti in testo. In questo caso, i numeri memorizzati nei file binari corrispondono alla codifica di numeri interi o in virgola mobile così come utilizzati nelle variabili del programma.

Ad ogni modo, al fine di conoscere quale decodifica utilizzare per un determinato file, si utilizza l'**estensione del nome del file** (per tutti i tipi di file, sia di testo che binari). L'estensione, infatti, è una sequenza ridotta di caratteri che appaiono alla fine del nome del file preceduti da un punto.

2.1 Operazioni sui file

Le operazioni principali di input/output (I/O) sui file sono due (Figura 1):

1. **scrittura** – i dati vengono memorizzati in un file (output file);
2. **lettura** – i dati vengono recuperati da un file (input file).

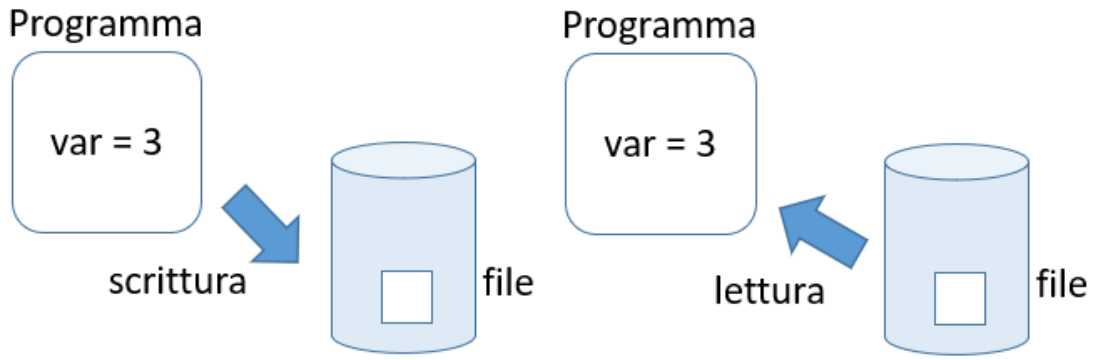


Figura 1. Schema della scrittura (a sinistra) e della lettura (a destra) da file

Ad ogni modo, per poter procedere con la scrittura o la lettura è necessario seguire i seguenti tre passi:

1. aprire il file (open);

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. elaborare il file (in lettura o scrittura);
3. chiudere il file (close).

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Accesso ai file

La gestione di un file in un programma comporta l'identificazione del file stesso mediante un nome. In programmazione vengono utilizzati **due nomi distinti**:

1. **nome fisico del file**, cioè il nome con cui il file viene salvato in memoria di massa che include l'estensione (es. prova.txt);
2. **nome logico del file**, cioè il nome che nel programma viene associato al nome fisico per poter interagire con esso.

   Il nome logico in Python è un **oggetto file** che rappresenta un vero e proprio gestore del file.

L'associazione tra nome fisico e nome logico di un file avviene all'**apertura di un file**. Per l'apertura di un file si utilizza la funzione **open**() il cui formato generale è:

```
oggetto_file = open(nomefile, modo)
```

dove ***nomefile*** è il nome fisico del file incluso il path (percorso del file in memoria di massa), mentre ***modo*** è una stringa con cui si specifica come verrà aperto il file:

- 'r' (read) per sola lettura;
- 'w' (write) per sola scrittura;
- 'a' per l'append.

Se la funzione open() riceve un *nomefile* che non contiene il path, si assume che il file risieda nella stessa directory del programma. Se il programma in esecuzione crea il file, il file viene creato nella stessa cartella del programma.

È comunque possibile specificare il **path** ed il *nomefile* tra i parametri della funzione open() utilizzando opportunamente il **prefisso r**. Il prefisso r indica che la stringa che si utilizzerà sarà una **stringa raw (grezza)** e fa in modo che l'interprete Python legga i caratteri di barra inversa come letterali di stringa come nell'esempio seguente:

```
test_file = open(r'C:\.....\text.txt','w')
```

È molto importante sottolineare il fatto che un file non può essere aperto sia in lettura sia in scrittura. Ad esempio, per poter leggere e scrivere un file si dovrà:

- aprire il file in lettura;
- leggere dal file;
- chiudere il file;
- aprire il file in scrittura;
- scrivere sul file;
- chiudere il file.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ipotizziamo quindi di voler scrivere dei dati su un file. Parlando di **oggetto file** e non di semplice variabile, dovremo utilizzare i **metodi** offerti dall'oggetto. I metodi sono delle funzioni che appartengono ad un oggetto e operano su di esso. Il metodo necessario per la scrittura su file è write () il cui formato generale è:

```
oggetto_file.write(string)
```

Al termine delle operazioni di scrittura (e di lettura) il file deve essere chiuso con il metodo **close**(), il cui formato è:

```
oggetto_file.close()
```

Si propone ora un esempio di scrittura su file di testo di indirizzi di posta di fantasia. È utile sottolineare che quando si apre un file in scrittura:

- se il file non esiste viene creato;
- se il file esiste esso viene cancellato completamente e si sovrascrive.

Il codice sorgente in Python che risolve il problema di scrittura su file è il seguente:

```
def main():
    outfile = open('indirizzi.txt','w')

    outfile.write("Via delle Margherite, 1\n")
    outfile.write("Via delle Peonie, 15\n")
    outfile.write("Via delle Libellule, 3\n")

    outfile.close()


main()
```

Ipotizziamo ora di voler leggere dati da un file. Uno dei metodi messi a disposizione da Python per la lettura da file è **read**() che restituisce in una stringa tutto ciò che è contenuto nel file di testo. Esiste anche il metodo **readline**() che legge un'intera linea dal file (restituisce una stringa incluso il carattere \n). Entrambi i metodi funzionano solo se il file è stato aperto in lettura.

Python mantiene automaticamente la **posizione di lettura**, cioè la locazione del prossimo elemento da leggere.

Si propone ora un esempio di lettura da file di testo di indirizzi di posta di fantasia (scritti su file di testo con il precedente esempio di scrittura) con la funzione read(). Anche in questo caso, è utile sottolineare che quando si apre un file in lettura, se il file non esiste viene sollevato un errore.

Il codice sorgente in Python che risolve il problema di lettura su file è il seguente:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    infile = open('indirizzi.txt','r')

    contenuto = infile.read()

    infile.close()

    print(contenuto)


main()
```

Il risultato dell'esecuzione del codice sorgente precedente è:

```
Via delle Margherite, 1
Via delle Peonie, 15
Via delle Libellule, 3
```

Si ripropone l'esempio precedente di lettura da file con la funzione readline(). Il codice sorgente corrispondente, la cui elaborazione produrrà lo stesso risultato appena presentato, è il seguente:

```
def main():
    infile = open('indirizzi.txt','r')

    riga1 = infile.readline()
    riga2 = infile.readline()
    riga3 = infile.readline()

    infile.close()

    print(riga1,end='')
    print(riga2,end='')
    print(riga3,end='')


main()
```

Quando, invece, si scrivono su file i valori referenziati dalle variabili del programma, conviene concatenare un carattere di newline prima di procedere alla scrittura con il metodo write() come, ad esempio, *outfile.write(nome + '\n')*.

In fase di lettura andrà poi utilizzato il metodo readline() che restituirà come stringa l'intera linea incluso il carattere di newline. A tal proposito interviene il **metodo rstrip() delle stringhe** per eliminare specifici caratteri alla fine della stringa.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il codice sorgente Python di esempio di concatenazione del carattere di newline in fase di scrittura su file:

```
def main():

    print("Inserisci i nomi delle tre fiabe preferite")
    f1 = input("Fiaba 1: ")
    f2 = input("Fiaba 2: ")
    f3 = input("Fiaba 3: ")

    outfile = open('fiabe.txt','w')

    outfile.write(f1 + '\n')
    outfile.write(f2 + '\n')
    outfile.write(f3 + '\n')

    outfile.close()



main()
```

Si riporta, ora, il codice sorgente Python di esempio di uso del metodo rstrip() per l'eliminazione del carattere di newline al termine della stringa in fase di lettura del file (il file di testo è quello scritto con il precedente programma):

```
def main():

    infile = open('fiabe.txt','r')

    riga1 = infile.readline()
    riga2 = infile.readline()
    riga3 = infile.readline()

    riga1 = riga1.rstrip('\n')
    riga2 = riga2.rstrip('\n')
    riga3 = riga3.rstrip('\n')

    infile.close()

    print(riga1)
    print(riga2)
    print(riga3)

main()
```

Il risultato dell'esecuzione dell'ultimo programma è:

```
Pinocchio
Cenerentola
Cappuccetto Rosso
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per evitare la sovrascrittura dei file in modalità scrittura 'w' è possibile utilizzare la modalità **append 'a'**. In modalità append, se il file esiste non viene cancellato, mentre se non esiste viene creato. I dati in append vengono, poi, aggiunti in coda al contenuto corrente del file.

Per poter scrivere valori numerici su file di testo, i numeri devono essere precedentemente convertiti in stringhe. Per tale scopo si utilizza la **funzione str()** che converte i valori passati come parametro in stringa. A loro volta, i numeri vengono letti come stringhe (stesso principio della funzione input). Anche in questo caso le funzioni **int()** e **float()** possono essere utilizzate per convertire stringhe in numeri in fase di lettura.

Si fornisce il codice sorgente di un programma che scrive su file tre variabili numeriche corrispondenti a tre votazioni:

```
def main():

    print("Inserisci le tre votazioni")
    v1 = int(input("Voto 1: "))
    v2 = int(input("Voto 2: "))
    v3 = int(input("Voto 3: "))

    outfile = open('voti.txt','w')

    outfile.write(str(v1) + '\n')
    outfile.write(str(v2) + '\n')
    outfile.write(str(v3) + '\n')

    outfile.close()

main()
```

L'input di esempio delle tre votazioni è il seguente:

```
Inserisci le tre votazioni
Voto 1: 9
Voto 2: 8
Voto 3: 9
```

Corrispondentemente, si riporta il codice sorgente del programma che effettua la lettura da file di variabili numeriche scritte utilizzando il precedente programma:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():

    infile = open('voti.txt','r')

    riga1 = infile.readline()
    riga2 = infile.readline()
    riga3 = infile.readline()

    riga1 = riga1.rstrip('\n')
    riga2 = riga2.rstrip('\n')
    riga3 = riga3.rstrip('\n')

    infile.close()

    v1 = int(riga1)
    v2 = int(riga2)
    v3 = int(riga3)

    print(v1,v2,v3)

main()
```

L'output corrispondente all'elaborazione del programma precedente è:

```
9 8 9
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Strutture iterative per l'elaborazione dei file

I file tipicamente vengono utilizzati per conservare grandi quantità di dati. Per tale motivo le strutture iterative possono essere usate in modo efficace per leggere e scrivere su file.

Spesso il numero di elementi memorizzati in un file non è noto, ma il metodo readline() restituisce una stringa vuota quando raggiunge **la fine del file EOF (End Of File)**.

In tal modo, è possibile scrivere un **ciclo while** con la condizione:

```
while line != ''
```

Si propone un programma alternativo a quello precedente per la lettura delle votazioni da file utilizzando il ciclo while:

```
def main():

    infile = open("voti.txt",'r')

    riga = infile.readline()

    while riga != '':
        print(riga)
        riga = infile.readline()

    infile.close()


main()
```

Ad ogni modo, Python permette ai programmatori di usare anche un **ciclo for** che legge automaticamente le righe in un file e termina quando viene raggiunta la fine del file (EOF). Il formato in Python di questa soluzione è:

```
for riga in oggetto_file:
    istruzioni
```

dove *riga* corrisponde ad un'intera riga del file di testo ad ogni iterazione del for.

Il programma corrispondente a quello precedente di lettura dei voti, ma che utilizza il ciclo for è il seguente:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():

    infile = open("voti.txt",'r')

    for riga in infile:
        print(riga)

    infile.close()


main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).